

K-Nearest Neighbors

Project-1:Tshirt Sizing

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: Tshirt=pd.read_csv('/Users/mujeebmohammed/Downloads/ML Classification Pac
Tshirt.head()
```

```
Out[2]:
```

	Height (in cms)	Weight (in kgs)	T Shirt Size
0	158	58	S
1	158	59	S
2	158	63	S
3	160	59	S
4	160	60	S

```
In [3]: X=Tshirt.iloc[:,0:2].values
X.shape
```

```
Out[3]: (18, 2)
```

```
In [4]: y=Tshirt.iloc[:,2].values
y
```

```
Out[4]: <StringArray>
['S', 'S', 'S', 'S', 'S', 'S', 'S', 'S', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L',
'L',
'L', 'L', 'L']
Length: 18, dtype: str
```

```
In [5]: #lets transform y values with 0 and 1 using labelcodes
from sklearn.preprocessing import LabelEncoder
LabelEncoder_y=LabelEncoder()
y=LabelEncoder_y.fit_transform(y)
y
```

```
Out[5]: array([1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [6]: #train_test_split
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_
```

```
In [7]: X_train.shape
```

```
Out[7]: (13, 2)
```

```
In [8]: X_test.shape
```

```
Out[8]: (5, 2)
```

```
In [9]: #model training
from sklearn.neighbors import KNeighborsClassifier
classifier=KNeighborsClassifier(n_neighbors=5,metric='minkowski',p=2)
classifier.fit(X_train,y_train)
```

```
Out[9]: ▼ KNeighborsClassifier ⓘ ?
        ► Parameters
```

```
In [10]: #model testing
y_pred=classifier.predict(X_test)
y_pred
```

```
Out[10]: array([1, 1, 0, 1, 0])
```

```
In [11]: y_test
```

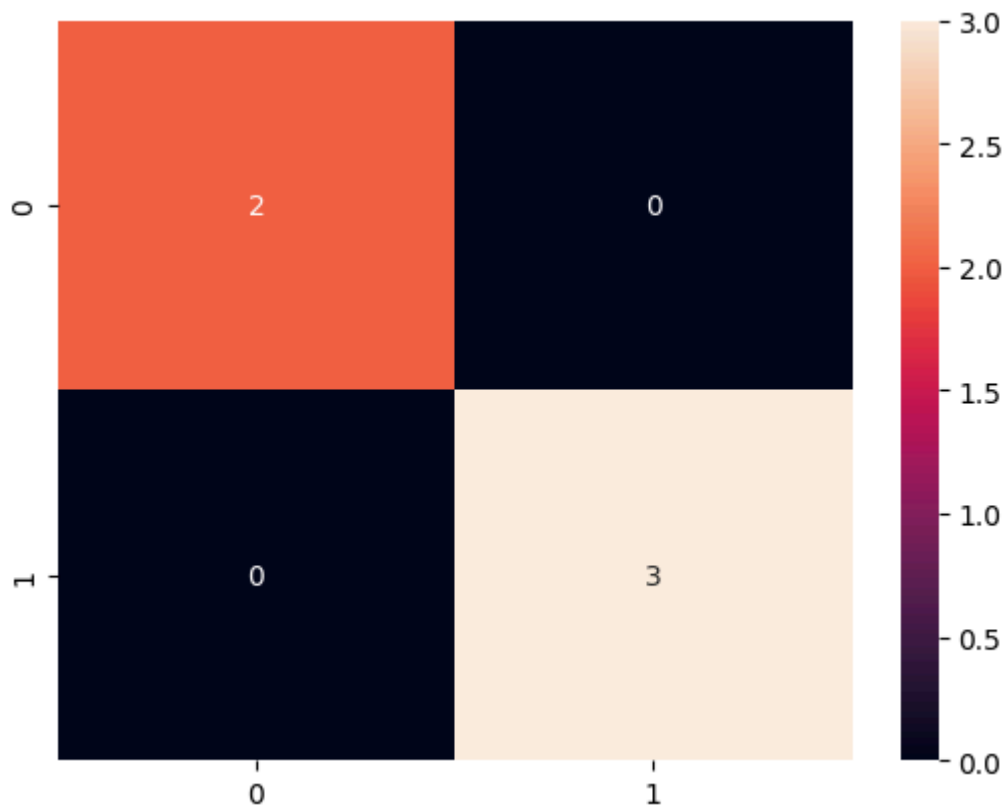
```
Out[11]: array([1, 1, 0, 1, 0])
```

```
In [12]: from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
cm
```

```
Out[12]: array([[2, 0],
               [0, 3]])
```

```
In [13]: sns.heatmap(cm,annot=True)
```

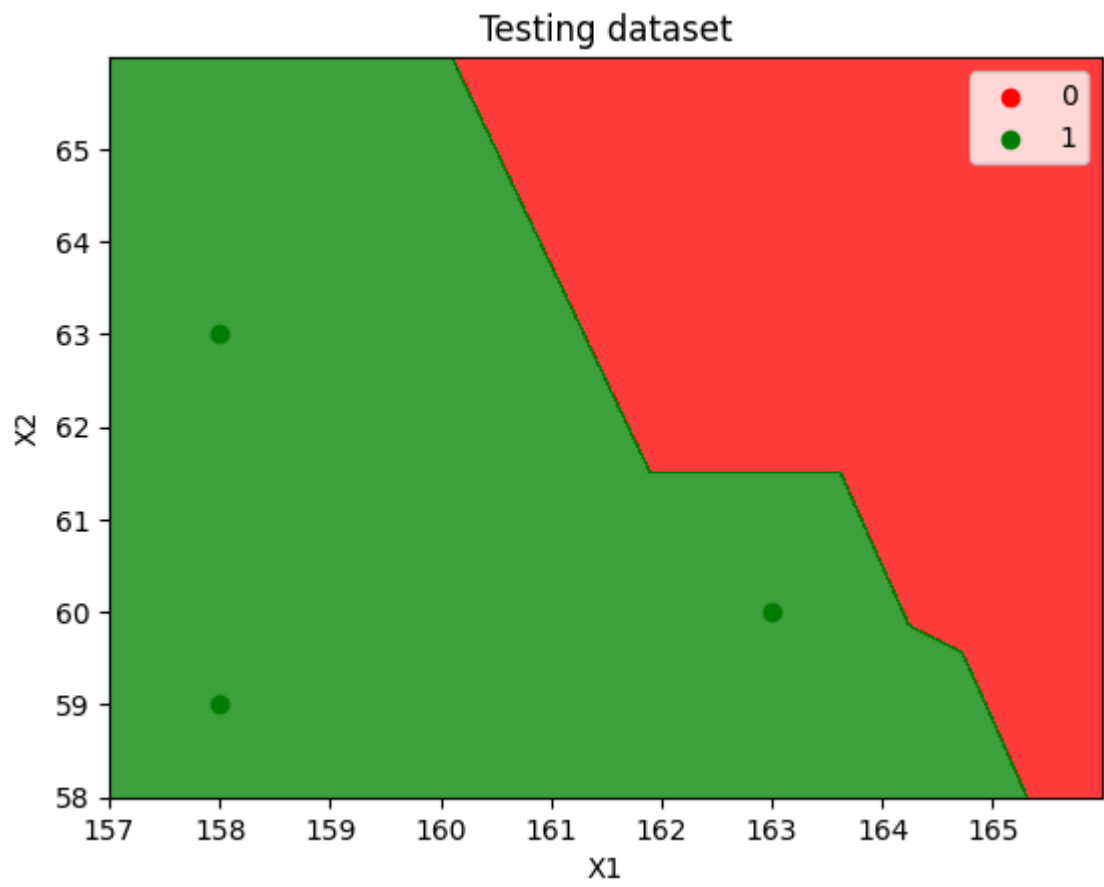
```
Out[13]: <Axes: >
```



```
In [15]: #visualising the testing set results
from matplotlib.colors import ListedColormap
X_grid,y_grid=X_test,y_test
X1,X2=np.meshgrid(
    np.arange(start=X_grid[:,0].min()-1,
               stop=X_grid[:,0].max()+1,
               step=0.01),
    np.arange(start=X_grid[:,1].min()-1,
               stop=X_grid[:,1].max()+1,
               step=0.01)
)
plt.contourf(X1,X2,classifier.predict(np.array([X1.ravel(),X2.ravel()]).T)
plt.xlim(X1.min(),X1.max())
plt.ylim(X2.min(),X2.max())
for i,j in enumerate(np.unique(y_grid)):
    plt.scatter(X_grid[y_grid==j,0],X_grid [y_grid==j,i],c=ListedColormap

plt.title('Testing dataset')
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend()
plt.show()
```

```
/var/folders/cs/gqgt5n4s34b8_mf66bwg0p1c0000gn/T/ipykernel_3409/387818518
3.py:17: UserWarning: *c* argument looks like a single numeric RGB or RGBA
sequence, which should be avoided as value-mapping will have precedence in
case its length matches with ** & *. Please use the *color* keyword-argu
ment or provide a 2D array with a single row if you intend to specify th
e same RGB or RGBA value for all points.
    plt.scatter(X_grid[y_grid==j,0],X_grid [y_grid==j,i],c=ListedColormap
(('red','green'))(i),label=j)
```



In []: